

`Kztek.Cameras.Avalonia` dispose khung hình ngay sau khi bắn sự kiện AI — subscriber đầu tiên luôn nhận ảnh rỗng

Mục	Nội dung
---	---
Ngày	2026-07-27
Project	DoorAlarmv3.Avalonia (cảnh báo camera AI)
Mức độ	HIGH — ảnh sự kiện mất, thất bại im lặng
Từ khoá	Kztek.Cameras, AIDetectAnalyzing, WriteableBitmap, ObjectDisposedException, async void

Bối cảnh

Port `FrmMain.Camera_AIDetectAnalyzing` sang Avalonia: đăng ký `camera.AIDetectAnalyzing`, khi AI phát hiện thì ghi `tblCameraAIEvent` và lưu ảnh sự kiện ra đĩa.

Điều thực tế xảy ra

Sự kiện bắn đúng, bản ghi vào CSDL đủ (61 sự kiện), nhưng **mọi file ảnh đều 0 byte**.

`WriteableBitmap.Save(path)` **không ném lỗi** — chỉ tạo file rỗng.

Thử sao chép pixel trước khi lưu thì lộ nguyên nhân thật:

```
System.ObjectDisposedException: Cannot access a disposed object.
```

Nguyên nhân

`AnvPlayerService` dispose ảnh **ngay trong `finally` sau `Invoke`**:

```
var fullFrameClone = AvaloniaFramePool.Clone(bitmap);
var boxedFrame = DrawRectWriteable(fullFrameClone, ...);
try
{
    AIDetectAnalyzing?.Invoke(this, new AIDetectAnalyzingEventArgs(...), boxedFrame);
}
finally
{
    // "Không có subscriber nào cho AIDetectAnalyzing trong toàn bộ app (đã kiểm tra)"
    boxedFrame.Dispose();
}
```

Giả định trong comment đúng **tại thời điểm viết**, và sai ngay khi có subscriber đầu tiên.

Handler `async void` trả về ở `await` đầu tiên ⇒ `finally` chạy ⇒ bitmap chết trong khi handler còn đang xử lý.

Cách khắc phục

Ở phía app (không sửa được thư viện dùng chung):

- Sao chép pixel **ngay dòng đầu tiên** của handler, trước cả một câu lệnh SQL.
- Không lấy được ảnh thì ghi `ImagePath rỗng`, đừng ghi đường dẫn tới file không tồn tại —

báo cáo về sau sẽ báo "không có ảnh" thay vì lỗi mở file.

3. Log **đúng một lần** (`Interlocked.Exchange`): sự kiện AI bắn liên tục, log mỗi lần sẽ nhấn chìm mọi log khác.

Sửa triệt để phải ở thư viện: đừng dispose trong `finally` khi có subscriber (dùng cờ đếm subscriber, hoặc chuyển quyền sở hữu cho subscriber).

Nguyên tắc rút ra

1. **Comment kiểu "đã kiểm tra, không ai dùng" là bom hẹn giờ.** Nó mô tả trạng thái codebase tại một thời điểm, không phải hợp đồng API.

2. Event handler nhận tài nguyên native (bitmap, buffer): coi như **chỉ hợp lệ trong phần đồng bộ** của handler. Có `await` là phải sao chép trước.

3. `WriteableBitmap.Save()` trên đối tượng đã dispose **tạo file 0 byte, không ném lỗi** — kiểm tra `FileInfo.Length` sau khi lưu nếu ảnh quan trọng.

Liên quan

- `[[kztek-cameras-start-signature-drift]]`
- `[[avalonia-writeablebitmap-binding-not-disposed-leak]]`